

1) Considerando como criterio de optimización a la minimización del transporte de información entre los nodos distribuidos (minimizar el tráfico de red). Sean las siguientes tablas de bases de datos distribuidas en varios nodos, pertenecientes a los alumnos de la Universidad del Nordeste:

Nodo 1: Alumnos				
LU	Apellido y Nombres	Dirección	Fecha de Nacimiento	Cód. de Localidad
6 bytes	40 bytes	40 bytes	8 bytes	4 bytes

Contiene: 14500 registros
Longitud del registro: 98 bytes

Nodo 2: Localidades		
Cód. de Localidad	Nombre de la localidad	Código de Provincia
4 bytes	45 bytes	2 bytes

Contiene: 4500 registros
Longitud del registro: 51 bytes

- a) Determinar el tamaño de las relaciones.
- b) Se desea obtener la siguiente consulta:
- Para cada alumno, obtener su Apellido y Nombres y el Nombre de la localidad a la cual pertenece.*
- i. Teniendo en cuenta el **Nodo 3**, como nodo resultado (nodo en el cual hay que entregar el resultado de la consulta), obtener las alternativas de solución a la consulta de tipo reunión (inner join) y optar por aquella solución que resulte más adecuada de acuerdo al criterio considerado (*"minimizar el transporte de bytes, entre los distintos nodos distribuidos"*).
 - ii. Si ahora consideramos al **Nodo 2**, como nodo resultado, obtener las alternativas de solución a la consulta de tipo reunión (inner join) y optar por aquella que resulte más óptima.
 - iii. Para la consulta planteada, en el punto anterior (**Nodo 2 como resultado**), procesar la consulta distribuida usando el tipo de semireunión (semijoin), obtener las posibles soluciones a la consulta.

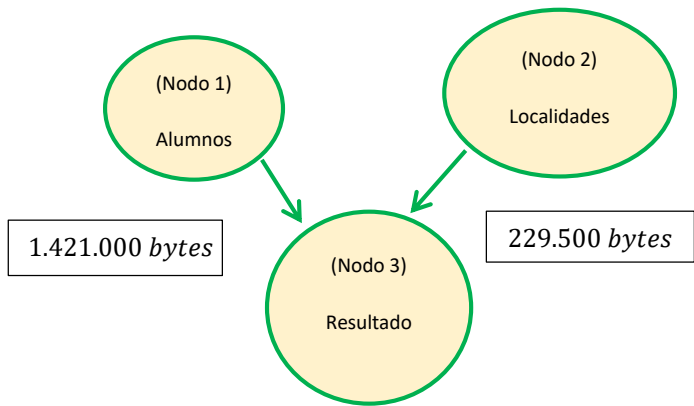
2) Determinar las instrucciones SQL necesarias, para la definición de las restricciones de integridad, reglas de negocio y triggers requeridos.

- a- Crear la tabla **Clientes** que contiene información relativa a las cajas de ahorros en pesos de una entidad bancaria, con los siguientes atributos: Nro. de cuenta de caja de ahorros, CUIL, Apellido y Nombres, código de localidad, situación crediticia y saldo, teniendo en cuenta lo siguiente:
- Nro. de cuenta de caja de ahorros: Clave primaria
 - Situación crediticia: puede contener los siguientes valores 'A', 'M', 'B'
 - Nro. de CUIL: tiene una integridad referencial con la tabla Afip(cuil)
 - Cód. de localidad: tiene una integridad referencial con la tabla Provincia(localidad)
- b- Crear la tabla **Debitos**, donde se registrarán aquellos importes que se debitarán de los saldos de las cajas de ahorros en pesos, sus atributos son: Nro. de cuenta de caja de ahorros, denominación, código de empresa e importe a debitar, teniendo en cuenta lo siguiente:
- Nro. de cuenta de caja de ahorros: Clave primaria
 - Código de empresa: tiene una integridad referencial con la Empresa(cod_empresa)
 - Contemplar una restricción de integridad (Constraint) de nombre "Control_Cuenta" de tal manera que nro. de cuenta de caja de ahorros, tenga una referencia externa a la tabla Clientes(nro_caja_ahorro)
- c- Crear la tabla **Clientes_de_baja**, donde se registrarán aquellos clientes dados de baja, con los siguientes atributos: Nro. de cuenta de caja de ahorros, usuario y fecha de baja.
- d- Crear un **trigger** de nombre **Baja_clientesB**, que permita insertar en la tabla **Clientes_de_baja**, los datos de aquel cliente cuya situación crediticia sea igual a 'B', donde las columnas usuario y fecha de baja de esta tabla, se grabarán con las variables del sistema USER y SYSDATE.
- El evento que dispara al trigger es **UPDATE** del atributo situación crediticia en la tabla Clientes, y su tiempo de acción es **BEFORE**.
- e- Crear un **trigger** de nombre **Debitos_pesos**, que permita descontar del saldo de la cuenta de caja de ahorros de la tabla Clientes, los importes de débitos que estén almacenados en la tabla Débitos para cada número de cuenta de caja de ahorros (saldo = saldo – importe a debitar).
- El evento que dispara al trigger es **INSERT** de la tabla Debitos y su tiempo de acción es **AFTER**.

Ejercicio 1:

- a) Tamaño de cada relación= Longitud de Registro *Cantidad de registros
- Teniendo en cuenta las longitudes y cantidades de tupas para los nodos Alumnos y Localidades.
- Alumnos (Nodo 1) contiene 14.500 registros con una longitud de registro de 98 bytes, y
- Localidades (Nodo 2) contiene 4.500 registros con una longitud de registro de 51 bytes.
- Calculamos:
- $$\text{Tamaño de Relación(Nodo 1: Alumnos)} = 98 \text{ bytes} \times 14.500 = 1.421.000 \text{ bytes}$$
- $$\text{Tamaño de Relación(Nodo 2: Localidades)} = 51 \text{ bytes} \times 4.500 = 229.500 \text{ bytes}$$
- b) Se desea obtener la siguiente consulta:
- $$Q1: \prod \text{Apellido y Nombres, Nombre de la localidad}(\text{Alumnos} * \text{Localidades})$$
- I. Teniendo en cuenta el Nodo 3, como nodo resultado (nodo en el cual hay que entregar el resultado de la consulta), obtener las alternativas de solución a la consulta de tipo reunión (inner join) y optar por aquella solución que resulte más adecuada de acuerdo al criterio considerado (“minimizar el transporte de bytes, entre los distintos nodos distribuidos”).
- El resultado de esta consulta constará de 14.500 tuplas y cada tupla resultante tendrá una longitud de (40 bytes del campo **Apellido y Nombres** de **ALUMNOS** + 45 bytes del campo **Nombre de la Localidad** de **LOCALIDAD**) 85 bytes. El tamaño del resultado será por tanto de **1.232.500 bytes**.

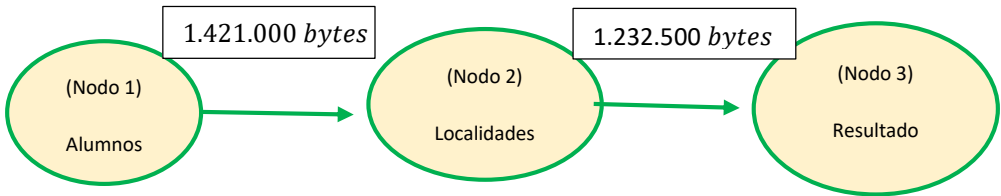
Opción 1:



Se transfiere tanto la relación Clientes, como la relación Sucursales al nodo Resultado (Nodo 3) y se realiza allí mismo la operación de Join. En este caso se transfieren:

$$1.421.000 \text{ bytes} + 229.500 \text{ bytes} = 1.650.500 \text{ bytes transferidos}$$

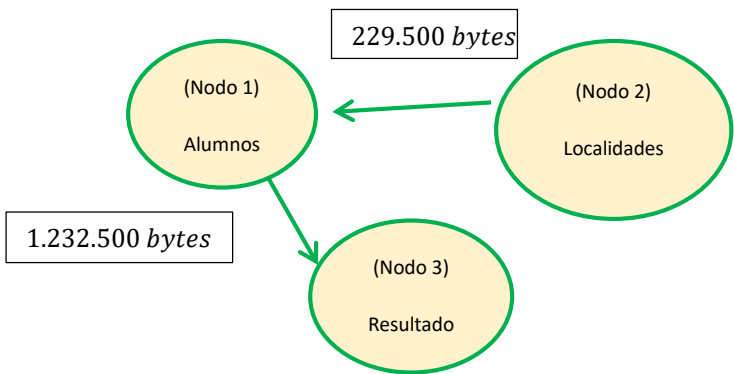
Opción 2:



Se transfiere la relación Clientes al nodo 2, ejecuta el join en este nodo y envía el resultado al nodo 3. Esto indica transferir:

$$1.421.000 \text{ bytes} + 1.232.500 \text{ bytes} = 2.653.500 \text{ bytes transferidos}$$

Opción 3:



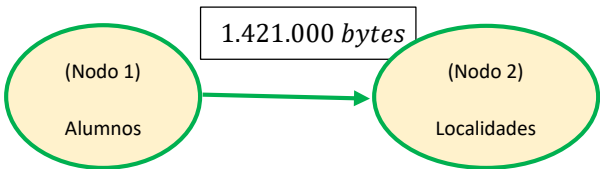
Se transfiere la relación Sucursales al nodo 1, ejecuta el join en este nodo y envía el resultado al nodo 3. En este caso, los bytes transferidos serían:

$$229.500\text{ bytes} + 1.232.500\text{ bytes} = 1.462.000\text{ bytes transferidos}$$

Conclusión: La Opción 3 es la mejor de entre las tres estrategias, dado que implica menos transferencia de datos por la red, cumpliendo así con el criterio de optimización solicitado para este ejercicio.

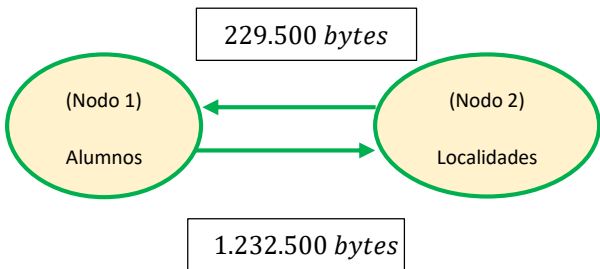
II. Si ahora consideramos al Nodo 2, como nodo resultado, obtener las alternativas de solución a la consulta de tipo reunión (inner join) y optar por aquella que resulte más óptima.

Opción 1:



Se transfiere la relación Clientes al nodo 2, se realiza el join y se presenta el resultado al usuario del nodo 2. De esta manera se transferirán: 1.421.000 bytes transferidos

Opción 2:



Se transfiere la relación Sucursales al nodo 1, se realiza el join y se envía el resultado al nodo 2. De esta manera se transferirán:

$$229.500\text{ bytes} + 1.232.500\text{ bytes} = 1.462.000\text{ bytes transferidos}$$

Conclusión: De las dos opciones evaluadas, la Opción 1 es la estrategia más adecuada, dado que implica menos transferencia de datos por la red, cumpliendo así con el criterio de optimización solicitado para este ejercicio.

III. Para la consulta planteada en el punto II, procesar la consulta distribuida usando el tipo de semi reunión (semi join), obtener las posibles soluciones a la consulta.

Paso 1: Proyección en Sucursales sobre atributos que van a intervenir en la operación de join y transferencia al nodo 1.

$$F1: \prod Cdo. Localidad (LOCALIDADES)$$

$$\text{Tamaño resultante} = 4\text{ bytes del atributo Cod. Localidad} \times 4.500\text{ tuplas de Localidades} = 18.000\text{ bytes transferidos}$$

Paso 2: Realización del join de las tuplas transferidas del paso anterior. Transferencia del resultado del join de nuevo al nodo 1. Se transfieren sólo los atributos necesarios para realizar el join final.

$$R1: \prod Cod. Localidades, Apellido_y_Nombres (F \times Alumnos)$$

$$\text{Tamaño } (4 + 40) \times 14.500 \text{ tuplas} = 638.000 \text{ bytes transferidos}$$

Sumando lo transferido en ambos pasos, tenemos un total de:

$$18.000 \text{ bytes} + 638.000 \text{ bytes} = 656.000 \text{ bytes transferidos}$$

Conclusión: La solución obtenida en el punto III es aún más óptima que la obtenida en ítem II.

Ejercicio 2:

```
CREATE TABLE Clientes (  
    Nro_Cuenta INTEGER PRIMARY KEY,  
    CUIL VARCHAR2(13) NOT NULL,  
    ApeYnom VARCHAR2 (30) NOT NULL,  
   Codigo_Localidad INTEGER NOT NULL,  
    Situacion_Crediticia CHAR (1) CHECK (Situacion_crediticia IN ('A', 'M', 'B')),  
    Saldo DECIMAL (10,2) NOT NULL,  
    CONSTRAINT fk_afip_cuil FOREIGN KEY (CUIL) REFERENCES Afip (cuil),  
    CONSTRAINT fk_provincia_localidad FOREIGN KEY (Codigo_Localidad) REFERENCES Provincia  
(localidad)  
);
```

```
CREATE TABLE Debitos(  
    Nro_cuenta INTEGER PRIMARY KEY NOT NULL,  
    Denominacion VARCHAR(100),  
    Cod_empresa INTEGER NOT NULL,  
    Importe_debitar DECIMAL(10, 2) NOT NULL,  
    CONSTRAINT Control_Cuenta FOREIGN KEY (Nro_cuenta) REFERENCES Clientes (Nro_cuenta),  
    CONSTRAINT fk_empresa FOREIGN KEY (Cod_empresa) REFERENCES Empresa (cod_empresa)  
);
```

```
CREATE TABLE Clientes_de_baja (  
    Nro_cuenta INTEGER PRIMARY KEY,  
    Usuario VARCHAR(50) NOT NULL,  
    Fecha_baja DATE NOT NULL,  
    CONSTRAINT fk_clientes_baja FOREIGN KEY (Nro_cuenta) REFERENCES Clientes (Nro_cuenta)  
);
```

```
CREATE OR REPLACE TRIGGER Baja_clientesB  
BEFORE UPDATE OF Situacion_crediticia ON Clientes  
FOR EACH ROW  
BEGIN  
    IF (:NEW.Situacion_crediticia = 'B') THEN  
        INSERT INTO Clientes_de_baja (Nro_cuenta, Usuario, Fecha_baja)  
        VALUES (:NEW.Nro_cuenta, USER, SYSDATE);  
    END IF;  
END;
```

```
CREATE OR REPLACE TRIGGER Debitos_pesos  
AFTER INSERT ON Debitos  
FOR EACH ROW  
BEGIN  
    UPDATE Clientes  
    SET Saldo = Saldo - :NEW.Importe_debitar  
    WHERE Nro_cuenta = :NEW.Nro_cuenta;  
END;
```